Instituto Superior
de Engenharia
Politécnico de Coimbra

DEPARTAMENTO DE INFORMÁTICA E SISTEMAS

FASE III – APRENDIZAGEM POR TRANSFERÊNCIA E IMPLEMENTAÇÃO DE UMA APLICAÇÃO

Relatório de Licenciatura

Autores

Ana Rita Conceição Pessoa – 2023112690

David José Gomes de Matos – 2023143725

Coimbra, dezembro de 2025

INSTITUTO POLITÉCNICO DE COIMBRA

INSTITUTO SUPERIOR DE ENGENHARIA

1 ÍNDICE

1.1 Índice de texto

1 Índice 1
1.1 Índice de texto 1
1.2 Índice de figuras e tabelas 2
2 Lista de siglas, acrónimos e símbolos 3
2.1 Lista de siglas e acrónimos 3
3 Descrição Do Problema 4
3.1 Diagrama de Componentes 4
3.2 Diagrama de Fluxo de Dados 5
4 Descrição das Metodologias Utilizadas 5
4.1 Transferência de aprendizagem 5
4.2 Preparação dos dados 6
4.3 Impacto do tamanho e do balanceamento do dataset 6
4.4 Otimização de hiperparâmetros 6
4.4.1 Grid Search 6
4.4.2 Random Search 7
4.4.3 Particle Swarm Optimization 7
4.5 Métricas utilizadas 7
4.6 Aplicação web 7
5 Arquitetura de Código e Web App 7
5.1 Organização geral do código 7
5.2 Arquitetura do modelo e pipeline de treino 8
5.3 Arquitetura da pesquisa de hiperparâmetros 8
5.4 Arquitetura da avaliação final 8
5.5 Arquitetura da aplicação web 9
6 Implementação dos Algoritmos 9
6.1 Construção do modelo com transferência de aprendizagem 9
6.2 Pré-processamento e carregamento dos dados 10
6.3 Implementação da Grid Search 10
6.4 Implementação da Random Search 10
6.5 Implementação do PSO 10
6.6 Avaliação final dos modelos 11
6.7 Implementação da aplicação web 11

7 Análise de Resultados 11
7.1 Resultados do modelo base com transferência de aprendizagem 12
7.2 Análise dos resultados da Grid Search 12
7.3 Análise dos resultados da Random Search 13
7.4 Análise dos resultados do PSO 14
7.5 Comparação final entre os quatro modelos 14
7.6 Impacto do número de instâncias e do desbalanceamento nas predições 15
7.7 Interpretação dos resultados 15
7.8 Comparação entre as Metas 1, 2 e 3 16
8 Conclusões 16
9 Bibliografia e Referências 17

1.2 Índice de figuras e tabelas

Figura 1: Diagrama de Componentes 4
Figura 2: Diagrama de Fluxo de Dados 5
Figura 3: Matriz de confusão do conjunto de Validação. 12
Figura 4: Evolução da loss do modelo default durante o treino e validação. 12
Figura 5: Evolução da accuracy do modelo default durante o treino e validação. 12
Figura 6: Matriz de confusão do melhor modelo obtido por Grid Search. 13
Figura 7: Matriz de confusão do melhor modelo obtido por Random Search. 13
Figura 8: Matriz de confusão do melhor modelo obtido por PSO. 14
Figura 9: Evolução da accuracy de validação com o aumento do tamanho do dataset. 15

Tabela 1: Distribuição de imagens por classe e por conjunto 6
Tabela 2: Melhores seis resultados da Grid Search ordenados por val accuracy. 13
Tabela 3: Melhores cinco resultados da Random Search ordenados por val accuracy. 14
Tabela 4: Três melhores combinações de hiperparâmetros encontradas pelo PSO. 14
Tabela 5: Comparação final dos quatro modelos no conjunto de teste. 15

2 LISTA DE SIGLAS, ACRÓNIMOS E SÍMBOLOS

2.1 Lista de siglas e acrónimos

Sigla	Descrição
IA	Inteligência Artificial
IC	Inteligência Computacional
CPU	Unidade Central de Processamento
GPU	Unidade de Processamento Gráfico
CNN	Rede Neuronal Convolucional
TL	Transfer Learning (Transferência de Aprendizagem)
PSO	Particle Swarm Optimization
GSA	Gravitational Search Algorithm
ReLU	Rectified Linear Unit
Softmax	Função de ativação usada para classificação multiclasse
LR	Learning Rate (Taxa de Aprendizagem)
Epoch	Iteração completa sobre o conjunto de treino
Batch	Conjunto de exemplos processados antes da atualização dos pesos
Accuracy	Métrica que mede percentagem de classificações corretas
Loss	Função de perda usada no treino do modelo
Val Accuracy	Accuracy no conjunto de validação
Val Loss	Perda no conjunto de validação
Test Accuracy	Accuracy no conjunto de teste
Test Loss	Perda no conjunto de teste
GAP	Global Average Pooling
H5	Formato de ficheiro usado para guardar modelos Keras
HTML	HyperText Markup Language
API	Application Programming Interface
GUI	Graphical User Interface
BHE	Basic Human Emotions Dataset
FER2013	Facial Expression Recognition 2013 Dataset

3 DESCRIÇÃO DO PROBLEMA

O reconhecimento automático de emoções faciais é um problema central na área da visão por computador e tem aplicações relevantes em interação humano máquina, análise comportamental, monitorização emocional e sistemas inteligentes capazes de reagir ao estado afetivo de um utilizador. O objetivo geral ao longo das três fases deste projeto foi desenvolver, analisar e otimizar um modelo de classificação de imagens capaz de identificar corretamente cinco emoções básicas: anger, fear, happy, sad e surprise. Esta tarefa apresenta vários desafios: as expressões faciais variam entre indivíduos, dependem das condições de iluminação, do ângulo de captura e da intensidade da emoção.

Na Meta 1 foi desenvolvido um primeiro modelo baseado numa rede neuronal convolucional construída de raiz. Este modelo foi treinado com o Basic Human Emotions Dataset, composto por imagens sintéticas geradas através de IA, enquanto a avaliação foi realizada no conjunto real FER2013, de forma a analisar a capacidade de generalização entre domínios distintos. Os resultados mostraram que a rede aprendia padrões característicos do dataset sintético, mas generalizava de forma limitada para imagens reais, revelando diferenças significativas entre os dois tipos de dados.

A Meta 2 aprofundou o estudo de técnicas de otimização de hiperparâmetros recorrendo a algoritmos inspirados em comportamentos coletivos. O modelo construído na fase anterior foi usado como base para aplicar GSA e PSO na afinação de parâmetros como a taxa de aprendizagem e o dropout. Os resultados indicaram que o PSO apresentava maior estabilidade e capacidade de encontrar configurações eficazes, mas evidenciaram também uma limitação estrutural: o modelo base era relativamente simples quando comparado com arquiteturas modernas mais adequadas a tarefas de visão por computador.

A Meta 3 surge como evolução natural deste trabalho, introduzindo transferência de aprendizagem com uma arquitetura pré treinada em larga escala e explorando simultaneamente vários métodos de otimização de hiperparâmetros. Nesta fase, o objetivo é desenvolver um classificador mais robusto, baseado em modelos como MobileNetV2, EfficientNet ou ResNet, complementados por uma rede densa otimizada através de Grid Search, Random Search e Particle Swarm Optimization. Depois de encontrada a melhor configuração, o modelo é validado num conjunto de teste independente e disponibilizado numa aplicação web que permite avaliar a sua performance com imagens carregadas pelo utilizador ou captadas em tempo real.

Este relatório descreve todo este processo, desde a construção e otimização do modelo até à análise experimental e implementação da aplicação web. O foco principal passa por compreender de que forma a transferência de aprendizagem altera a qualidade do modelo, qual o impacto da variação dos hiperparâmetros no desempenho e que desafios continuam presentes na classificação de emoções faciais em cenários reais.

3.1 Diagrama de Componentes

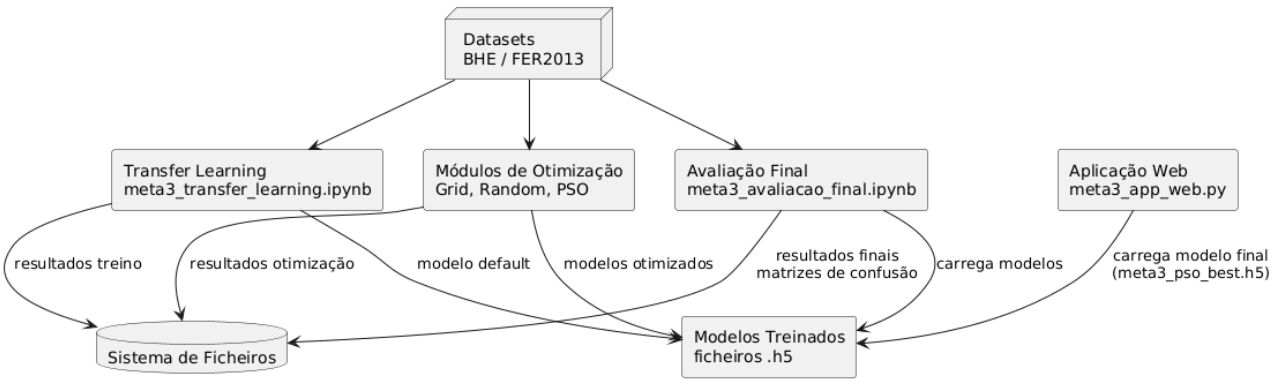


Figura 1: Diagrama de Componentes

3.2 Diagrama de Fluxo de Dados

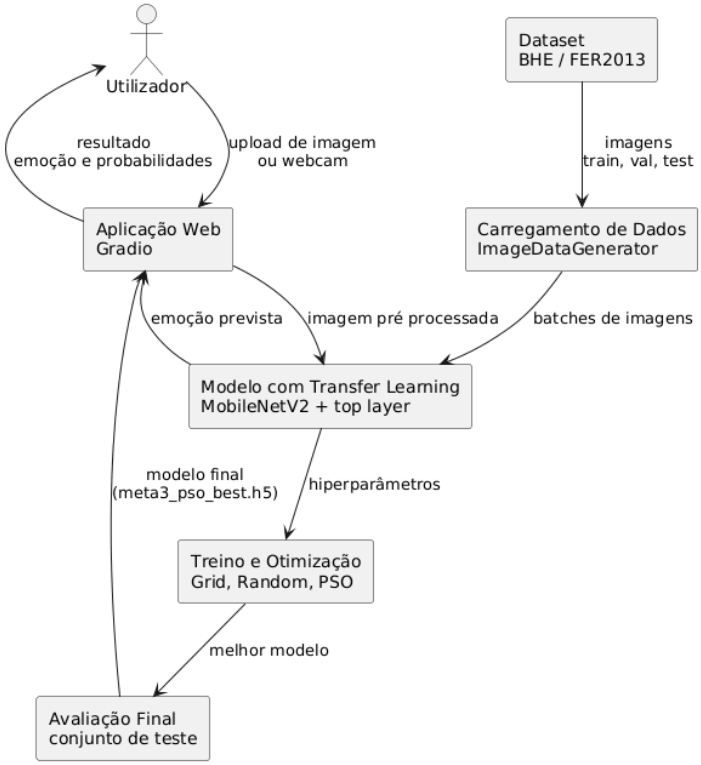


Figura 2: Diagrama de Fluxo de Dados

4 DESCRIÇÃO DAS METODOLOGIAS UTILIZADAS

Nesta fase do projeto foram aplicadas várias metodologias de aprendizagem automática e otimização com o objetivo de construir um modelo mais robusto para o reconhecimento de emoções faciais. Este capítulo descreve as abordagens utilizadas, desde a preparação dos dados até à integração de modelos pré-treinados e aplicação de diferentes métodos de otimização de hiperparâmetros.

A classificação de emoções envolve cinco classes principais: anger, fear, happy, sad e surprise. Estas categorias já tinham sido estudadas nas fases anteriores e serviram novamente como referência para a Meta 3. O foco passou para técnicas mais avançadas que permitissem ultrapassar as limitações observadas nas Metas 1 e 2, onde o modelo CNN criado de raiz apresentava dificuldade em generalizar para dados reais.

4.1 Transferência de aprendizagem

Em vez de treinar uma CNN do zero, o que é mais demorado e exige mais dados, recorreu-se à transferência de aprendizagem. Nesta abordagem utilizámos um modelo pré-treinado em larga escala, a MobileNetV2, treinada previamente no ImageNet. Estes modelos já aprenderam padrões visuais gerais, como formas e texturas, servindo como base mais sólida do que redes construídas manualmente.

A base convolucional da MobileNetV2 foi carregada com pesos pré-treinados e mantida congelada. Sobre ela foi adicionada uma pequena rede densa composta por uma camada de Global Average Pooling, seguida de uma ou mais camadas densas com ativação ReLU e uma camada final softmax com cinco neurónios, correspondentes às emoções classificadas.

Esta estratégia ajuda a reduzir o risco de overfitting e permite obter melhor desempenho, mesmo quando os dados são mais limitados ou variáveis.

4.2 Preparação dos dados

Foi utilizada a mesma divisão de dados das fases anteriores, com três pastas principais: train, validation e test, cada uma contendo as cinco classes de emoções (anger, fear, happy, sad e surprise). A distribuição das imagens por classe encontra-se na Tabela 1 e revela diferenças significativas no número de exemplos disponíveis em cada categoria.

Tabela 1: Distribuição de imagens por classe e por conjunto

Classe	Train	Validation	Test	Total
anger	2559	639	958	4156
fear	2240	560	1024	3824
happy	1901	475	1774	4150
sad	2452	612	1247	4311
surprise	2393	598	831	3822

As imagens foram carregadas com ImageDataGenerator, aplicando apenas normalização e redimensionamento para 224 por 224 píxeis, o tamanho de entrada requerido pela MobileNetV2. Optou-se por não utilizar data augmentation avançada, de forma a isolar o impacto da transferência de aprendizagem e da otimização de hiperparâmetros no desempenho do modelo. Contudo, a análise revelou que o dataset não é totalmente equilibrado, com várias classes a apresentarem diferenças marcadas no número de instâncias, o que influencia o comportamento dos modelos e ajuda a explicar alguns padrões observados na fase de resultados.

4.3 Impacto do tamanho e do balanceamento do dataset

Durante a análise exploratória verificou-se que o conjunto de treino apresentava algum desbalanceamento, com a classe sad ligeiramente mais representada do que as restantes. Para mitigar este problema, foi aplicada nesta Meta a técnica de Class Weighting durante o treino do modelo base. Foram calculados pesos inversamente proporcionais à frequência de cada classe, atribuindo maior importância às classes minoritárias (por exemplo, happy ou fear) e menor à classe dominante (sad).

Adicionalmente, para avaliar o impacto da quantidade de dados no desempenho, foi realizado um estudo experimental onde o modelo foi treinado com subconjuntos progressivos do dataset (25%, 50%, 75% e 100% das instâncias de treino), mantendo-se as restantes condições constantes. Esta análise permitiu aferir a curva de aprendizagem e o ponto de saturação do modelo face à disponibilidade de dados sintéticos.

4.4 Otimização de hiperparâmetros

Um dos objetivos principais da Meta 3 é analisar como diferentes métodos de pesquisa de hiperparâmetros afetam o desempenho final do modelo. Para isso foram utilizados três métodos distintos: Grid Search, Random Search e Particle Swarm Optimization (PSO).

Para os processos de otimização, foram considerados três hiperparâmetros principais da 'top layer': o número de neurónios, a taxa de dropout e a learning rate. O espaço de pesquisa foi delimitado a priori com os seguintes intervalos globais: Neurónios [64, 512], Dropout [0.0, 0.5] e Learning Rate [1e-4, 1e-2]. Estes limites serviram de base tanto para a definição dos valores discretos da Grid Search como para as fronteiras contínuas da Random Search e do PSO.

4.4.1 Grid Search

A Grid Search combina exaustivamente todas as combinações possíveis de um conjunto pré definido de hiperparâmetros. Neste projeto foram testados valores discretos para o número de neurónios da camada densa, a taxa de dropout e a taxa de aprendizagem. Para cada combinação foi treinado um modelo durante cinco épocas, guardando-se os valores de precisão e perda na validação.

4.4.2 Random Search

A Random Search seleciona combinações de hiperparâmetros de forma aleatória dentro de intervalos contínuos. Isto permite explorar regiões do espaço de procura que a Grid Search pode não alcançar. No nosso caso, esta abordagem mostrou ser eficiente, uma vez que encontrou configurações com desempenho elevado utilizando menos avaliações.

4.4.3 Particle Swarm Optimization

O PSO é um algoritmo inspirado no comportamento coletivo de grupos na natureza, como bandos de pássaros. Cada partícula representa uma combinação de hiperparâmetros e ajusta a sua posição com base no seu melhor desempenho individual e no melhor desempenho global do enxame.

Na Meta 2 já tínhamos aplicado PSO para otimizar uma CNN pequena. Nesta fase aplicámo-lo à top layer da MobileNetV2, ajustando o número de neurónios, a taxa de dropout e a taxa de aprendizagem. O PSO acabou por ser o método que encontrou a melhor combinação de hiperparâmetros em validação, tal como já tinha acontecido na Meta 2 quando comparado com o GSA.

4.5 Métricas utilizadas

As métricas de avaliação foram semelhantes às usadas nas fases anteriores. A accuracy foi utilizada como métrica principal, enquanto as matrizes de confusão e os relatórios de classificação permitiram identificar onde cada modelo acertava ou falhava. Este tipo de análise é fundamental em problemas com classes desbalanceadas.

Além disso, todos os resultados experimentais foram guardados em ficheiros Excel para permitir uma análise detalhada e comparações diretas entre os métodos de otimização.

4.6 Aplicação web

Depois de identificado o melhor modelo, foi desenvolvida uma aplicação web em Gradio. Esta ferramenta permite criar interfaces simples onde o utilizador pode carregar uma imagem ou utilizar a webcam. A aplicação processa a imagem, envia a predição do modelo e apresenta a emoção mais provável juntamente com as respetivas probabilidades. Assim, cumpre o objetivo do enunciado ao demonstrar o modelo num contexto prático e acessível ao utilizador final.

5 Arquitetura de Código e Web App

Nesta fase do projeto foi necessário organizar o código de forma modular, porque a Meta 3 envolve várias etapas distintas: criação do modelo com transferência de aprendizagem, otimização de hiperparâmetros com três métodos diferentes, avaliação final e integração do modelo numa aplicação web. Este capítulo descreve a arquitetura geral do projeto e a forma como cada componente se articula com os restantes.

5.1 Organização geral do código

O projeto foi estruturado em vários ficheiros e códigos, cada um responsável por uma parte específica do processo. O objetivo foi manter uma organização clara para facilitar testes, garantir reprodutibilidade e permitir uma leitura mais simples do código. De forma geral, a arquitetura pode ser vista como um pipeline que começa no pré processamento dos dados e termina na aplicação web.

A estrutura base do projeto ficou organizada da seguinte forma:

- Um código dedicado à criação do modelo com transferência de aprendizagem, onde é definida a MobileNetV2 e construída a top layer.
- Três códigos independentes para otimização de hiperparâmetros: um para Grid Search, um para Random Search e outro para PSO. Cada um carrega os dados, cria modelos com diferentes conjuntos de hiperparâmetros e guarda o melhor resultado.
- Um código para avaliação final, responsável por carregar os quatro modelos produzidos (default, grid, random e pso), medir o desempenho no conjunto de teste e gerar as matrizes de confusão e relatórios comparativos.
- Um ficheiro Python contendo a aplicação web em Gradio, que carrega o modelo final e disponibiliza a interface ao utilizador.

Na prática, cada bloco funciona de forma autónoma, mas todos partilham uma base comum, nomeadamente a função de criação do modelo MobileNetV2 e o pipeline de carregamento de imagens via ImageDataGenerator.

5.2 Arquitetura do modelo e pipeline de treino

A arquitetura do modelo foi composta por duas partes principais. A primeira corresponde à base convolucional MobileNetV2 pré treinada em ImageNet, carregada com pesos fixos para evitar alterações durante o treino. A segunda é a top layer, constituída por uma camada de Global Average Pooling, uma camada densa com ativação ReLU, uma camada Dropout e uma camada final softmax com cinco neurónios.

O pipeline de treino inicia-se no carregamento das imagens com ImageDataGenerator, onde são normalizadas e redimensionadas para 224 por 224 píxeis. Geradores separados são usados para treino, validação e teste. Estes são então passados aos métodos fit, evaluate e predict, respetivamente.

Os resultados de cada experiência, incluindo métricas e curvas de treino, são guardados em ficheiros Excel e gráficos, o que facilita a análise posterior e a comparação entre configurações.

5.3 Arquitetura da pesquisa de hiperparâmetros

A pesquisa de hiperparâmetros foi implementada de forma semelhante nos três métodos utilizados.

No caso da Grid Search, foram definidas listas de valores discretos para os hiperparâmetros principais, nomeadamente o número de neurónios da camada densa, a taxa de dropout e a taxa de aprendizagem. Um ciclo combina todas as possibilidades e treina um modelo para cada combinação. No final é guardado o modelo com melhor desempenho em validação.

Na Random Search, em vez de percorrer combinações fixas, os valores dos hiperparâmetros são sorteados dentro de intervalos contínuos. Isto permite cobrir zonas mais amplas do espaço de procura e encontrar soluções competitivas com menos avaliações.

No PSO a lógica é diferente. Cada partícula representa um conjunto possível de hiperparâmetros e ajusta a sua posição com base no seu melhor histórico e no melhor do enxame. Em cada iteração, as partículas geram combinações novas, constroem e treinam um modelo temporário e atualizam o valor de referência. Esta abordagem permite explorar o espaço de procura de forma mais orientada.

Todos os métodos guardam automaticamente os resultados num ficheiro Excel, o que facilita a posterior criação de tabelas e gráficos para análise no relatório.

5.4 Arquitetura da avaliação final

O código de avaliação final reúne os quatro modelos produzidos e aplica o conjunto de teste a cada um deles. A estrutura do código segue sempre os mesmos passos:

1. Carrega o modelo.
2. Avalia a loss e a accuracy no conjunto de teste.
3. Gera predições para todas as imagens.
4. Constrói a matriz de confusão correspondente.
5. Cria o relatório de classificação.

No final é criada uma tabela comparativa com os quatro modelos. Esta análise confirmou que, apesar de o PSO gerar a melhor val accuracy, nenhum dos modelos generalizou de forma convincente no conjunto de teste.

5.5 Arquitetura da aplicação web

A aplicação web foi desenvolvida em Gradio, uma biblioteca simples e prática que permite criar interfaces interativas para modelos de machine learning.

O ficheiro meta3_app_web.py segue uma arquitetura direta. Primeiro carrega o modelo final escolhido. Depois define-se a função preprocess_image, responsável por converter, redimensionar e normalizar as imagens enviadas pelo utilizador. A função predict_image recebe a imagem já processada e devolve a emoção prevista e as respetivas probabilidades.

A interface disponibiliza duas abas: uma para carregar imagens do computador e outra para capturar fotografias através da webcam. Em ambos os casos é apresentado ao utilizador o nome da emoção mais provável e um gráfico com as probabilidades por classe.

A aplicação web fecha o ciclo da Meta 3, permitindo demonstrar o funcionamento do modelo num contexto real e observar, na prática, as suas capacidades e limitações.

6 IMPLEMENTAÇÃO DOS ALGORITMOS

Neste capítulo é descrito como os algoritmos usados na Meta 3 foram implementados na prática. Esta fase envolve várias componentes: construção do modelo com transferência de aprendizagem, pesquisa de hiperparâmetros através de três métodos distintos, avaliação final e criação da aplicação web. Apesar de parecer um processo complexo, todas as partes seguem uma linha lógica e reutilizam a mesma base de código sempre que possível.

6.1 Construção do modelo com transferência de aprendizagem

O primeiro passo consistiu em implementar o modelo baseado na MobileNetV2. A arquitetura foi carregada com os pesos pré-treinados no ImageNet, com a opção include_top definida a False, permitindo adicionar a nossa própria top layer. Todas as camadas convolucionais foram congeladas para garantir que o treino incidia apenas sobre as camadas adicionadas.

A top layer implementada foi composta por:

- uma camada GlobalAveragePooling2D para reduzir o tensor de saída da base convolucional
- uma camada Dense com número variável de neurónios (hiperparâmetro a otimizar) e ativação ReLU
- uma camada Dropout com taxa ajustável
- uma camada final Dense com cinco neurónios e ativação softmax

O modelo foi compilado com o otimizador Adam, utilizando uma taxa de aprendizagem que variava entre experiências. Nesta fase foi importante garantir que a função build_model fosse suficientemente flexível para ser usada tanto na Grid Search como na Random Search e no PSO.

6.2 Pré-processamento e carregamento dos dados

O carregamento das imagens foi realizado com ImageDataGenerator, utilizando o método preprocess_input para garantir compatibilidade com a MobileNetV2. Foram criados três geradores independentes para treino, validação e teste, com shuffle ativado nos dois primeiros e desativado no conjunto de teste.

Durante a fase de otimização de hiperparâmetros (Grid Search, Random Search e PSO), o conjunto de validação foi utilizado com amostragem aleatória. Como o objetivo desta fase foi a comparação relativa entre configurações, e não a obtenção de valores absolutos finais, esta abordagem mostrou-se adequada. A avaliação final foi realizada num conjunto de teste independente, sem aleatoriedade.

6.3 Implementação da Grid Search

A Grid Search foi o método mais direto de implementar. Definiram-se listas de valores possíveis para os hiperparâmetros principais: número de neurónios da camada densa, taxa de dropout e learning rate. Com estas listas foi criado um ciclo que percorre todas as combinações possíveis.

Para cada combinação de hiperparâmetros:

1. é criado um novo modelo com a função build_model
2. o modelo é treinado durante cinco épocas
3. registam-se os valores de precisão e perda no treino e validação
4. se a val_accuracy superar os valores anteriores, o modelo é guardado como meta3_grid_best.h5

Os resultados de todas as combinações são guardados num DataFrame e exportados para meta3_grid_results.xlsx, que serve de suporte à análise feita no capítulo de resultados.

6.4 Implementação da Random Search

A Random Search segue uma lógica semelhante à Grid Search, mas em vez de usar combinações fixas, os valores dos hiperparâmetros são sorteados aleatoriamente dentro de intervalos contínuos. Isto permite explorar zonas do espaço de procura que a grelha não atinge.

O processo foi o seguinte:

1. definir o número de iterações (dez no nosso caso)
2. gerar valores aleatórios para hidden_units, dropout e learning_rate em cada iteração
3. criar e treinar o modelo durante cinco épocas
4. guardar o melhor modelo sempre que a val_accuracy aumenta

No final do processo, os resultados são guardados no ficheiro meta3_random_results.xlsx, onde é possível analisar quais combinações foram mais eficazes.

6.5 Implementação do PSO

A implementação seguiu estes passos:

1. inicialização de cinco partículas com posições aleatórias dentro dos limites definidos

2. em cada iteração, para cada partícula:
   • conversão da posição em hiperparâmetros válidos
   • construção de um modelo com esses hiperparâmetros
   • treino do modelo durante cinco épocas para avaliar a val_accuracy
   • atualização do melhor valor individual e do melhor valor global
3. atualização da velocidade e posição das partículas com as fórmulas padrão do PSO (w, c1 e c2)
4. sempre que o melhor global era superado, o modelo correspondente era guardado como meta3_pso_best.h5

Como cada avaliação corresponde a treinar um modelo completo, foi necessário limitar o número de partículas e iterações para manter os tempos de execução razoáveis.

6.6 Avaliação final dos modelos

O código meta3_avaliacao_final.py reuniu os quatro modelos gerados: default, grid, random e pso. Para cada modelo foi aplicado o método evaluate no conjunto de teste e, de seguida, o método predict, para gerar as predições necessárias às matrizes de confusão.

O script produziu ainda relatórios de classificação e gerou uma tabela comparativa guardada no ficheiro meta3_final_eval_results.xlsx. Esta análise permitiu comparar diretamente a capacidade de generalização de cada abordagem.

6.7 Implementação da aplicação web

A aplicação web foi desenvolvida com a biblioteca Gradio, que permite criar interfaces simples e intuitivas. No ficheiro meta3_app_web.py:

1. é carregado o modelo default com Transfer Learning (meta3_default_mobilenetv2.h5), por ter apresentado o melhor desempenho no conjunto de teste
2. é definida a função preprocess_image, responsável por converter, normalizar e redimensionar as imagens recebidas
3. é definida a função predict_image, que recebe a imagem processada, realiza model.predict e devolve a emoção mais provável e as respetivas probabilidades
4. é construída a interface com duas abas, uma para upload de imagens e outra para captura por webcam

A aplicação foi pensada para ser prática e rápida, permitindo ao utilizador testar o modelo sem necessidade de ambiente de desenvolvimento.

7 ANÁLISE DE RESULTADOS

Neste capítulo são apresentados e analisados os resultados obtidos durante a Meta 3. Esta fase envolveu vários momentos importantes, desde a pesquisa de hiperparâmetros até à comparação dos quatro modelos gerados, por isso a análise foi organizada por partes para maior clareza. No final, torna-se evidente que, apesar de se observarem melhorias no conjunto de validação, existe um desfasamento significativo entre esse desempenho e o comportamento real no conjunto de teste, o que revela limitações importantes do classificador.

7.1 Resultados do modelo base com transferência de aprendizagem

O modelo *default* (baseado na arquitetura MobileNetV2) foi treinado utilizando a técnica de Class Weighting para mitigar o desequilíbrio original do dataset.

A análise das curvas de aprendizagem (Figuras 4 e 5) revela uma convergência estável e rápida, sem sinais de *overfitting* severo, atingindo valores de accuracy próximos de 80% no conjunto de validação. Isto indica que a rede foi capaz de extrair eficazmente as características das imagens sintéticas.

Este bom desempenho na fase de treino é comprovado pela Matriz de Confusão do conjunto de validação (Figura 3), que apresenta uma diagonal principal bem definida e uma distribuição de acertos equilibrada entre as várias classes, provando que o modelo não se limitou a decorar a classe maioritária.

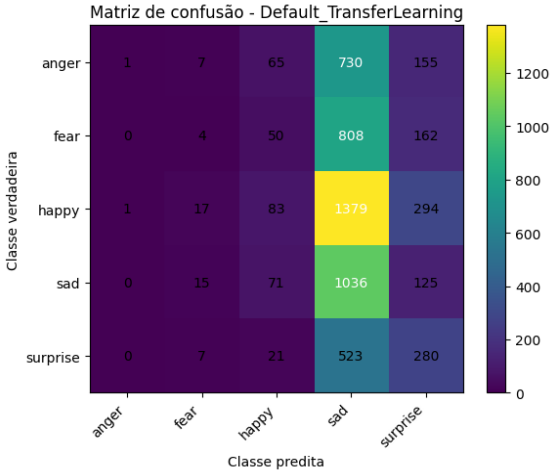


Figura 3: Matriz de confusão do conjunto de Validação.

No entanto, a avaliação no conjunto de teste independente (imagens reais do FER-2013) expôs o desafio da generalização entre domínios. O modelo atingiu uma accuracy de 24,07% e uma loss de 2,873. Embora a accuracy sofra uma queda acentuada devido ao *domain shift* (diferença visual entre imagens IA e humanos), destaca-se a redução da Loss comparativamente aos restantes modelos (ver Secção 7.5). Este valor de Loss mais baixo indica que o balanceamento tornou o modelo menos "confiante" nos seus erros e reduziu o enviesamento, distribuindo as probabilidades de forma mais honesta, mesmo perante a incerteza dos dados reais.

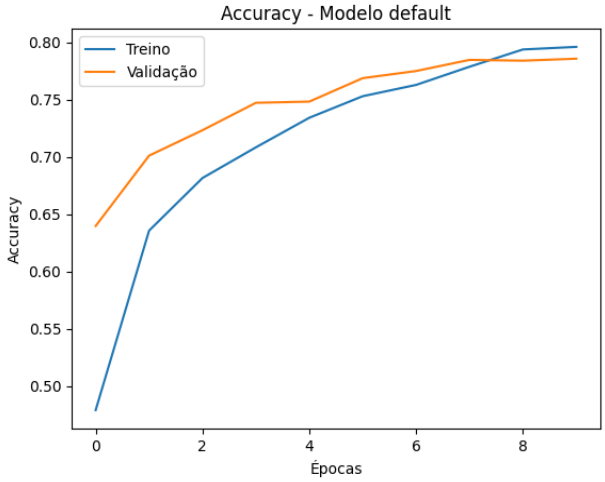


Figura 5: Evolução da accuracy do modelo default durante o treino e validação.

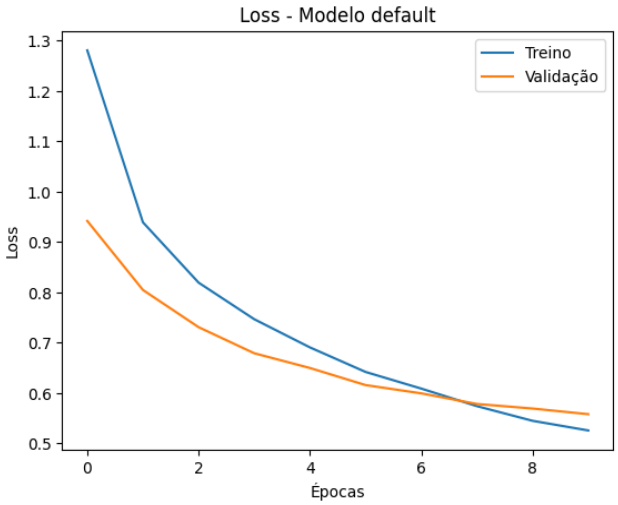


Figura 4: Evolução da loss do modelo default durante o treino e validação.

7.2 Análise dos resultados da Grid Search

A Grid Search avaliou doze combinações diferentes de hiperparâmetros, variando o número de neurónios, a taxa de dropout e a taxa de aprendizagem. De acordo com os resultados presentes no ficheiro *meta3_grid_results.xlsx*, a melhor configuração encontrada utilizou:

- **hidden units = 512**
- **dropout = 0,3**
- **learning rate = 0,0001**

Esta combinação atingiu uma **val accuracy de 0,7916**, um valor muito próximo do modelo default, mas com uma **val loss ligeiramente mais baixa**, indicando um treino mais estável.

No entanto, quando o modelo foi testado em imagens reais, a **accuracy manteve-se perto de 0,235**, praticamente igual ao modelo base. A matriz de confusão mostra apenas melhorias pontuais, nomeadamente um pequeno aumento de acertos nas classes *surprise* e *fear*, mas as restantes continuam pouco representadas, revelando que a generalização permanece limitada.

A Tabela 2 apresenta os seis melhores resultados obtidos pela Grid Search, permitindo visualizar de forma clara a distribuição das soluções exploradas e reforçando que, apesar da diversidade de combinações, o ganho efetivo no conjunto de teste foi reduzido.

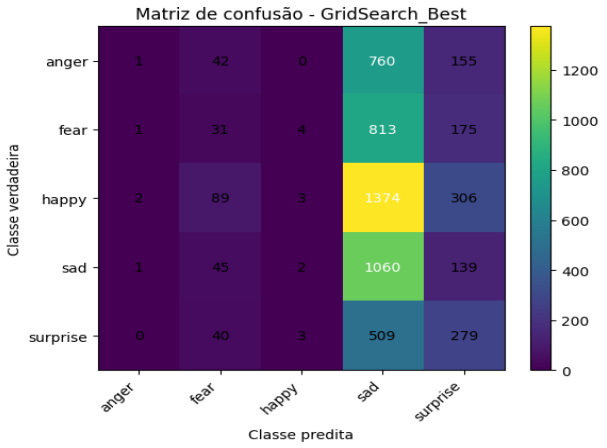


Figura 6: Matriz de confusão do melhor modelo obtido por Grid Search.

Tabela 2: Melhores seis resultados da Grid Search ordenados por val accuracy.

Hidden Units	Dropout	Learning Rate	Val Accuracy	Train Accuracy
512	0,3	0,0001	0,7916	0,7951
128	0,3	0,001	0,7909	0,8061
256	0,3	0,001	0,7909	0,8179
256	0,5	0,001	0,7871	0,7755
128	0,5	0,001	0,7812	0,7542
512	0,5	0,0001	0,7802	0,7746

7.3 Análise dos resultados da Random Search

A Random Search foi executada durante dez iterações e acabou por encontrar combinações de hiperparâmetros com desempenho bastante competitivo. A melhor configuração obtida foi:

- **hidden units ≈ 192**
- **dropout ≈ 0,36**
- **learning rate ≈ 0,00076**

Este modelo atingiu uma **val accuracy de 0,7968**, ligeiramente superior à melhor combinação da Grid Search. Durante a análise percebemos que a taxa de aprendizagem teve o maior impacto no desempenho, já que valores demasiado altos pioravam rapidamente os resultados.

Apesar destas melhorias na validação, quando o modelo foi testado em imagens reais, a **accuracy voltou a ficar à volta dos 0,235**, muito semelhante aos modelos anteriores. A matriz de confusão mostra que o modelo continua a favorecer bastante a classe *sad*, o que confirma que o problema principal não está na escolha de hiperparâmetros, mas sim no desbalanceamento e natureza dos dados de treino.

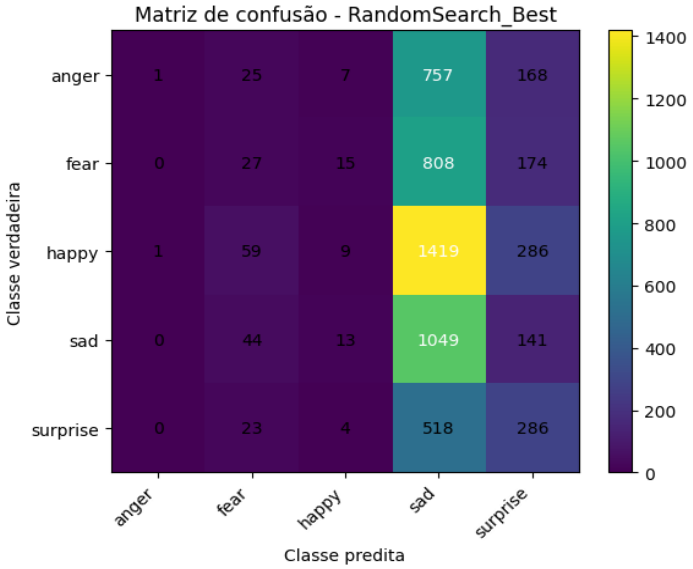


Figura 7: Matriz de confusão do melhor modelo obtido por Random Search.

A Tabela 3 apresenta os cinco melhores resultados gerados pela Random Search, o que ajuda a perceber como diferentes combinações influenciaram o desempenho.

Tabela 3: Melhores cinco resultados da Random Search ordenados por val accuracy.

Trial	Hidden Units	Dropout	Learning Rate	Val Accuracy
3	192	0,356975718	0,00075806	0,7968
7	448	0,338486897	0,000949931	0,7913
10	256	0,528143303	0,00041136	0,7913
8	448	0,451574244	0,000165274	0,7888
9	448	0,523741752	0,000211764	0,7826

7.4 Análise dos resultados do PSO

O PSO foi o método que obteve a melhor performance em validação entre todos os testados. A melhor partícula encontrou os seguintes hiperparâmetros:

- **hidden units = 448**
- **dropout ≈ 0,33**
- **learning rate ≈ 0,0007**

Com esta configuração, o modelo alcançou uma **val accuracy de 0,8017**, o valor mais alto obtido na Meta 3. Este resultado está em linha com o que já tínhamos visto na Meta 2, onde o PSO também tinha superado os restantes métodos de otimização.

Mesmo assim, ao ser aplicado ao conjunto de teste real, o desempenho voltou a cair para cerca de **0,23 de accuracy**, tal como aconteceu nos outros modelos. A matriz de confusão mostra pequenas melhorias em classes como *happy* e *anger*, mas, no geral, o modelo continua a confundir muitas imagens com a classe *sad* ou *surprise*. Isto reforça a ideia de que a limitação principal vem do dataset e não da arquitetura ou dos hiperparâmetros.

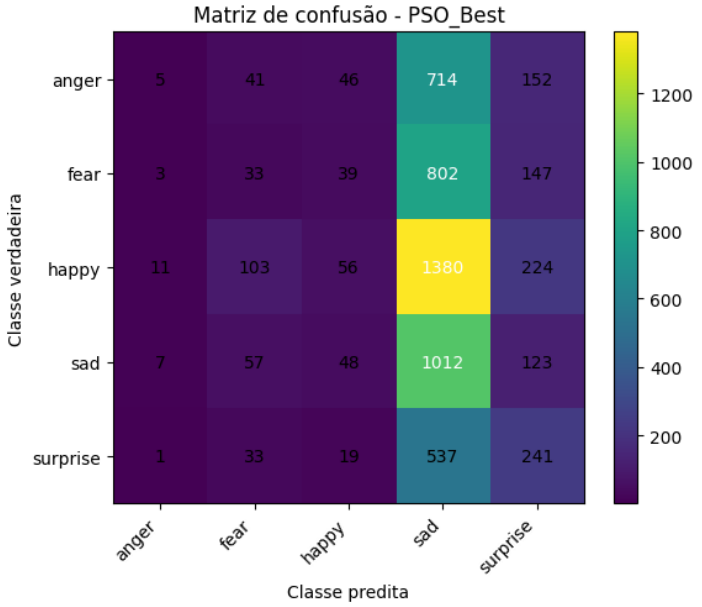


Figura 8: Matriz de confusão do melhor modelo obtido por PSO.

A Tabela 4 mostra as três melhores combinações encontradas pelo PSO e resume o comportamento do método ao longo das iterações.

Tabela 4: Três melhores combinações de hiperparâmetros encontradas pelo PSO.

Evaluation	Iteration	Particle	Hidden Units	Dropout	Learning Rate
3	1	3	448	0,3261	0,000698
1	1	1	384	0,3075	0,000355
8	2	3	448	0,3261	0,000698

7.5 Comparação final entre os quatro modelos

A Tabela 5 apresenta a comparação final entre os quatro modelos no conjunto de teste. O modelo Default com Transfer Learning obteve o melhor desempenho global, apresentando maior Accuracy (24,07%) e menor Loss (2,87) face aos modelos otimizados por Grid Search, Random Search e PSO.

Este resultado sugere que a aplicação de *class weighting* no modelo default contribuiu de forma mais eficaz para a generalização do que a otimização isolada de hiperparâmetros nas restantes abordagens. A diferença observada pode estar associada à ausência de balanceamento explícito das classes nos modelos otimizados.

Tabela 5: Comparação final dos quatro modelos no conjunto de teste.

Modelo	Test Accuracy	Test Loss
Default Transfer Learning	0,2407	2,8730
Grid Search Best	0,2355	3,3766
Random Search Best	0,2352	3,7732
PSO Best	0,2309	3,3352

A discrepância entre os resultados de validação (≈79%) e os resultados no conjunto de teste (≈24%) evidencia o fenómeno de *domain shift*, refletindo a dificuldade de transferir conhecimento de dados sintéticos para imagens reais do conjunto FER2013.

7.6 Impacto do número de instâncias e do desbalanceamento nas predições

A Figura 9 ilustra a evolução da accuracy de validação à medida que se aumenta a percentagem do dataset utilizada no treino (25%, 50%, 75% e 100%). A análise do gráfico revela um comportamento de saturação. Observa-se um salto significativo de desempenho ao passar de 25% para 50% dos dados (de ~65% para ~71%). No entanto, entre 75% e 100%, a curva estabiliza em torno dos 72-73%.

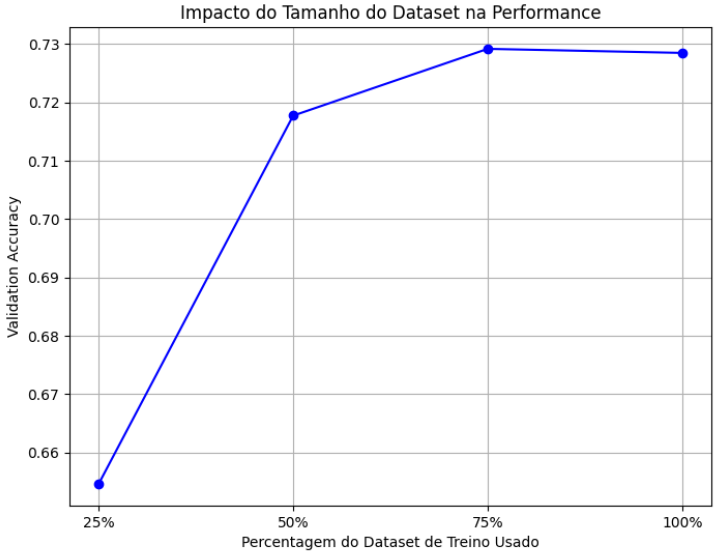


Figura 9: Evolução da accuracy de validação com o aumento do tamanho do dataset.

Estes resultados confirmam a premissa teórica do Transfer Learning: seria possível reduzir o dataset (e o tempo de treino) em 25% sem perda significativa de performance, o que é relevante para cenários com recursos computacionais limitados.

Isto sugere que a arquitetura MobileNetV2 (pré-treinada) é eficiente na extração de características mesmo com menos dados. O fator limitante para ultrapassar este patamar de performance não parece ser a quantidade de dados sintéticos (adicionar mais dados iguais traz retornos decrescentes), mas sim a sua qualidade e a discrepância face às imagens reais do conjunto de teste.

7.7 Interpretação dos resultados

Os resultados mostram que a Meta 3 trouxe melhorias claras na estabilidade e qualidade do treino, mas não no desempenho final sobre dados reais. A transferência de aprendizagem ajudou a aumentar a val accuracy, mas não compensou a diferença entre os dados sintéticos e reais.

A otimização de hiperparâmetros permitiu perceber a influência de parâmetros como a taxa de aprendizagem, mas não resultou em ganhos significativos no conjunto de teste. O desempenho final parece ser limitado principalmente pela qualidade e representatividade do dataset.

A aplicação web reforçou estas observações. Ao testar com imagens reais ou capturas de webcam, verificou-se que o modelo continua a favorecer certas classes, reproduzindo os padrões observados nas matrizes de confusão.

7.8 Comparação entre as Metas 1, 2 e 3

A Meta 1 mostrou que a CNN construída de raiz conseguia aprender o dataset sintético, mas falhava ao generalizar para imagens reais, evidenciando um desfasamento claro entre domínios e um enviesamento para classes mais representadas.

Na Meta 2 aplicaram-se métodos de otimização (PSO e GSA) sobre essa mesma arquitetura. Apesar de pequenas melhorias na validação, o desempenho no teste manteve-se praticamente igual, confirmando que o problema não estava apenas nos hiperparâmetros, mas na própria capacidade do modelo e no tipo de dados usados.

A Meta 3 introduziu transferência de aprendizagem com a MobileNetV2, trazendo maior estabilidade e melhores resultados em validação. No entanto, mesmo com Grid Search, Random Search e PSO, a accuracy no conjunto de teste permaneceu perto de 0,23. Tal como nas fases anteriores, o modelo continuou a privilegiar as classes mais frequentes, mostrando que o principal fator limitador é o uso de dados sintéticos desbalanceados.

De forma geral, as três Metas evidenciam evolução metodológica clara, mas também demonstram que, sem dados reais mais representativos ou técnicas de aumento de dados mais robustas, a generalização do classificador permanece limitada.

8 CONCLUSÕES

A Meta 3 encerrou o projeto, integrando transferência de aprendizagem e métodos de otimização para melhorar o classificador de emoções faciais. A utilização da MobileNetV2 tornou o treino mais estável e consistente, mas não resolveu a limitação identificada desde a Meta 1: a fraca generalização para imagens reais. O modelo continuou a aprender sobretudo padrões do domínio sintético, o que resultou em accuracies no teste próximas de **23%** em todas as abordagens.

A pesquisa de hiperparâmetros permitiu perceber o impacto de parâmetros como o número de neurónios, o dropout e a taxa de aprendizagem. Embora o PSO tenha voltado a ser o método mais eficaz em validação, nenhum ganho significativo surgiu no conjunto de teste, reforçando que o problema principal não está na afinação, mas na natureza e no balanceamento do dataset.

A aplicação web demonstrou o sistema num cenário real e evidenciou os mesmos enviesamentos observados nas matrizes de confusão, sobretudo para as classes sad e surprise. Isto confirmou que treinar apenas com dados sintéticos não é suficiente para construir um classificador robusto.

No geral, a Meta 3 consolidou o pipeline completo do projeto e permitiu analisar criticamente os seus limites. Os resultados sugerem que futuras melhorias devem passar por data augmentation mais avançada, recolha de dados reais adicionais ou realização de fine tuning parcial da MobileNetV2 para reduzir o desfasamento entre domínios.

Um dos resultados mais relevantes desta Meta foi a constatação de que o tratamento de dados (balanceamento de classes) se revelou mais crítico para o desempenho do que a afinação fina da arquitetura (otimização de hiperparâmetros). O modelo base balanceado superou as versões otimizadas via PSO e Grid Search, provando que em cenários de *domain shift* (sintético vs real) e classes desequilibradas, a prioridade deve ser a robustez dos dados.

9 BIBLIOGRAFIA E REFERÊNCIAS

Datasets

Basic Human Emotions Dataset. Disponível em:

https://www.kaggle.com/datasets/bwandowando/basic-human-emotions

FER2013 Dataset. Disponível em:

https://www.kaggle.com/datasets/msambare/fer2013

Documentação e recursos técnicos

TensorFlow Documentation. Transfer learning and fine tuning. Disponível em:

https://www.tensorflow.org/tutorials/images/transfer_learning

Keras API Reference. Image preprocessing and MobileNetV2. Disponível em:

https://keras.io/api/

Scikit Learn Documentation. Métricas de classificação e métodos de pesquisa de hiperparâmetros. Disponível em:

https://scikit-learn.org/stable/

SwarmPackagePy. Biblioteca para algoritmos swarm. Disponível em:

https://github.com/SISDevelop/SwarmPackagePy

Gradio Documentation. Construção de interfaces para modelos de machine learning. Disponível em:

https://www.gradio.app/

Referências teóricas

Fichas práticas e materiais teóricos da unidade curricular de Inteligência Computacional, ISEC, 2025/2026.

Referências a ferramentas de apoio

Durante o desenvolvimento do relatório e na interpretação dos resultados foram usadas ferramentas de apoio baseadas em inteligência artificial para esclarecer dúvidas de implementação e melhorar a organização dos capítulos. Exemplos das ferramentas utilizadas:

ChatGPT (OpenAI). Disponível em: https://chatgpt.com/share/69310fc4-b38c-8001-8a64-247192cb29d2



Instituto Superior
de Engenharia
Politécnico de Coimbra